

16782 HW1

Contents

1 Introduction

In this homework, we explore real-time planning algorithms for 8 connected grid environments.

2 Problem

2.1 Goal

A target moves one step at a time in an 8-connected grid environment. Its trajectory is known to the robot prior to the start of the planning process. The goal is to catch the target while accumulating as little cost as possible.

2.2 Environment

The environment is an 8-connected grid deterministic and fully observable. The robot knows the costs and occupancy of all cells in the environment at all times including the target's trajectory.

The environment can be as large as 2000×2000 cells, while the target trajectory can span around 5500 time steps. A full A* search over this space is infeasible because its worst-case complexity is $O(b^d)$, where b is the branching factor and d is the search depth. Here, $b = 9$, since the robot may move to any neighboring cell or remain still, and d can reach 5500. Therefore, A* may expand up to 9^{5500} nodes in the worst case, which is computationally impractical.

On top of this computational challenge the robot must plan in real-time as the target is moving, so the planner must be able to make decisions quickly and efficiently.

3 Methodology

3.1 Problem Formulation

The environment is represented as an 8-connected grid graph with valid cells V . A robot state is $s = (p, t)$, where $p = (x, y) \in V$ is its position and t is the current time step. Each action moves the robot to a neighboring cell or keeps it in place, advances time by one step, and has a traversal cost $c(p, p')$.

The known target trajectory is $Q = (q_0, q_1, \dots, q_T)$, where $q_\tau = (x_\tau, y_\tau)$ is the target position at time τ and T is the final time step. An interception occurs when the robot reaches q_τ at the same time τ . The objective is to find a valid interception path $\pi = (p_t, \dots, p_\tau)$ that minimizes

$$J(\pi) = \sum_{k=t}^{\tau-1} c(p_k, p_{k+1}), \quad p_\tau = q_\tau.$$

For convenience, let $\mathcal{Q} = \{q_\tau \mid 0 \leq \tau \leq T\}$ denote the set of all cells visited by the target.

3.2 Planning Algorithms

In this problem, the target is moving in real time and the robot must catch it before it reaches the end of its trajectory. Therefore, a full A* with a consistent heuristic or Dijkstra search is impractical, unless the environment was small enough that the full search would finish before the target takes its first step. However this is not the cause in this problem.

Some Real-Time planning algorithms solve this by doing a bounded A* search from the robot's current state at each time step to decide on the next action the robot should take and instead of throwing the entire search away they do a learning step in which they make the heuristics more informed for future searches. Such algorithms which was implemented in this work is Learning Real-Time A* (LRTA*) and Real-Time Adaptive A* (RTAA*).

The two algorithms differ in the learning step: LRTA* updates the heuristic by dynamic programming (in this work backward dijkstra), while RTAA* by a linear pass.

LRTA* makes the heuristic more informed but it takes more time per learning step compared to RTAA*, which performs a simpler linear update.

Below is the pseudocode for the two algorithms used in this work: LRTA* and RTAA*.

3.3 Heuristic

The heuristic combines three components:

1. **Earliest interception:** Estimates the earliest possible interception using Chebyshev distance.
2. **Departure feasibility:** Checks whether interception is still possible before the trajectory ends.
3. **Spatial cost:** Estimates the minimum cost to the nearest target position while accounting for obstacles and traversal costs.

3.3.1 Earliest Interception

The earliest-interception heuristic gives the exact number of time steps needed to reach the target in a relaxed 3D space with no obstacles and a uniform traversal cost of 1. The Chebyshev

distance to a target position is

$$d_{\infty}((x, y), (x_{\tau}, y_{\tau})) = \max(|x - x_{\tau}|, |y - y_{\tau}|).$$

The heuristic is then

$$h_{EI}(s) = \min_{t \leq \tau \leq T} \{\tau - t \mid \tau - t \geq d_{\infty}((x, y), (x_{\tau}, y_{\tau}))\}.$$

If no time step satisfies the condition, the heuristic returns ∞ .

Since this is the exact cost-to-go for the relaxed problem, the heuristic is consistent and provides a lower bound on the cost of the original problem.

A direct approach checks each future target position until the interception condition is met, taking $O(N)$ time in the worst case for N remaining positions. However, since the target moves by at most one cell per time step, once the condition becomes true, it remains true for all later time steps. Therefore, binary search can find the earliest feasible interception time in $O(\log N)$.

3.3.2 Departure Feasibility

This is a state feasibility check rather than a cost-to-go estimate. It determines whether interception is still possible from a given state.

Let $d(p, q_{\tau})$ be the minimum number of valid moves from p to q_{τ} while accounting for obstacles. Since each move takes one time step and the robot can wait, interception at time τ is possible when

$$t \leq \tau - d(p, q_{\tau}).$$

The latest feasible departure time from p is therefore

$$L(p) = \max_{0 \leq \tau \leq T} \{\tau - d(p, q_{\tau})\},$$

A state is infeasible when $t > L(p)$.

This objective can be rewritten as a minimization by defining $D(p) = T - L(p)$:

$$D(p) = T - L(p) = \min_{0 \leq \tau \leq T} \{(T - \tau) + d(p, q_{\tau})\}.$$

This can be computed using a backward multi-source Dijkstra search over the 2D grid. Each target position q_{τ} is initialized with $T - \tau$, and each reverse move costs 1. Using the Dijkstra cost g , $D(p)$ is evaluated during the search as

$$D(p) = \begin{cases} g(p), & p \in \text{CLOSED}, \\ \min_{u \in \text{OPEN}} g(u), & p \notin \text{CLOSED}, \text{ OPEN} \neq \emptyset, \\ \infty, & p \notin \text{CLOSED}, \text{ OPEN} = \emptyset. \end{cases}$$

The feasibility value is then

$$h_{\text{DF}}(s) = \begin{cases} \infty, & D(p) > T - t, \\ 0, & \text{otherwise.} \end{cases}$$

3.3.3 Spatial Cost

The spatial-cost heuristic gives the exact cost of reaching the lowest-cost target cell while accounting for obstacles and traversal costs in the relaxed 2D space (x, y) . For position p , the heuristic is

$$h_{\text{SC}}(p) = \min_{\substack{\pi=(p_0, \dots, p_k) \\ p_0=p, p_k \in \mathcal{Q}}} \sum_{i=0}^{k-1} c(p_i, p_{i+1}).$$

Since the time dimension is ignored, this value is a consistent lower bound on the actual cost in the full 3D space. It is computed using a backward multi-source Dijkstra search initialized with every cell in $\mathcal{Q}$. Using the Dijkstra cost g , the heuristic is then defined as

$$h_{\text{SC}}(p) = \begin{cases} g(p), & p \in \text{CLOSED}, \\ \min_{u \in \text{OPEN}} g(u), & p \notin \text{CLOSED}, \text{ OPEN} \neq \emptyset, \\ \infty, & p \notin \text{CLOSED}, \text{ OPEN} = \emptyset. \end{cases}$$

At any fixed stage of the search, this definition is consistent.

3.3.4 Heuristic Evaluation

The overall heuristic used is the maximum of the three consistent heuristics defined above as follows:

$$h(s) = \max\{h_{\text{DF}}(s), h_{\text{SC}}(p), h_{\text{EI}}(s)\}.$$

This overall heuristic remains consistent as it is the maximum of consistent heuristics.

4 Results

5 Conclusion

6